

FP8GCN: An Edge-FPGA-Based Graph Convolutional Network Accelerator with FP8 Approximate Multipliers

Shizhen Huang, *Member, IEEE*, Jiaren Peng , Yanxiang Zhu , *Member, IEEE*, and Han Bao 

Abstract—GCN accelerators often adopt pruning or quantization to reduce resource demands, yet energy efficiency remains a key challenge on FPGA platforms due to DSP-based matrix multiplications. Existing DSP-free approximate multipliers fail to strike a good trade-off between precision and efficiency. To overcome these, this letter presents the first FPGA-based FP8 approximate GCN accelerator, enabling both low precision loss and high energy efficiency. The proposed architecture is deployed on a Kintex-7 K325T FPGA, demonstrating that the accuracy loss of the approximate FP8 GCN model remains within 3%. While achieving average performance improvements of $12.9\times$ ($0.7\text{--}44.0\times$) and average energy savings of $169\times$ ($1.01\text{--}1010\times$) over CPUs, GPUs, and prior edge-FPGA-based accelerators.

Index Terms—GCN, Approximate multipliers, FPGA, Hardware accelerator, 8-bit floating-point.

I. INTRODUCTION

GRAPH Convolutional Networks (GCNs) have demonstrated strong capability in processing graph-structured data and have been widely applied in domains such as recommendation systems, intelligent transportation, and device intrusion detection [1]. However, as the scale of graph data grows and model complexity increases, GCNs inevitably incur substantial computation and memory access overheads, posing significant challenges for efficient inference on resource-constrained edge devices.

To enable efficient edge deployment of GCN inference for embedded applications, researchers have explored various optimization strategies and developed a range of hardware accelerators. LW-GCN [2] introduces a novel compression format to optimize workload distribution and adopts a quantization strategy that uses SINT16 for features and weights, and SINT4 for non-zero elements, significantly reducing memory consumption. BoostGCN [3] proposes an innovative feature aggregation scheme to reduce memory traffic and overall memory access latency. ATE-GCN [4] accelerates GCN inference by asymmetrical ternary quantization and a high-frequency unified PE array for efficient DSP utilization.

Although existing accelerators have achieved notable progress in certain aspects, there remains a room for further optimization, particularly for edge applications with stringent

resource and energy constraints. Previous designs depend heavily on DSP-based matrix operations [5], which are energy-intensive and incompatible with many edge FPGAs that offer little to no DSP resources (e.g. Lattice iCE40 and Gowin GW1N). While designing DSP-free approximate multipliers [6] is an effective strategy to balance energy efficiency and accuracy, most previous GCN implementations utilize INT8 quantization. Integrating INT8-based approximate multipliers [7]–[9] into GCN accelerators might lead to considerable accuracy degradation. Therefore, achieving a balance between power consumption and precision loss is critical for enabling efficient GCN inference deployment on edge devices to meet the demands of embedded applications.

To address the aforementioned challenges, this letter proposes FP8GCN, an edge-FPGA-based GCN accelerator with FP8 approximate multipliers. The main contributions of this letter are summarized as follows. **1.** We quantize the feature matrix, weight matrix, and adjacency matrix in the GCN model using the FP8 format, which significantly reduces memory access overhead while preserving data fidelity. **2.** We design a fully pipelined FPGA architecture with an FP8 approximate computation core. This architecture performs approximate operations on FP8 data and adopts a DSP-free design to significantly reduce power consumption—while considerably mitigating the accuracy degradation often observed in prior approximate designs. **3.** We deploy the proposed accelerator on a Kintex-7 K325T FPGA board to validate its effectiveness. To the best of our knowledge, this is the first implementation of FP8 quantization for GCNs. The proposed quantization and approximation framework achieves a favorable trade-off between accuracy and energy efficiency during GCN inference.

II. PROPOSED FP8GCN

A. FP8 quantization

The FP8 format is composed of a sign bit, exponent bits, and mantissa bits. It is typically represented in either the E4M3 or E5M2 format, where E refers to the number of exponent bits and M refers to the number of mantissa bits,

$$x_{\text{DEC}} = (-1)^S \times 2^E \times M, \quad (1)$$

where $E = e - \text{bias}$ and $M = 1 + m$. The *bias* in this context varies with the number of bits in the exponent and is determined by the following formula:

$$\text{bias} = 2^{e-1} - 1. \quad (2)$$

Note that an implicit 1, namely the *hiddenbit*, is concatenated to the fraction as an integer bit and forms the significand. A

Received 6 June 2025; revised xx xxxxx 2025; accepted xx xxxxx 2025. Date of publication xx xxxxx xxxxx; date of current version xx xxxxx xxxxx.

Shizhen Huang and Jiaren Peng are with the School of Physics, Information Engineering and Microelectronics, Fuzhou University, Fuzhou 350108, China (email: hs501@qq.com, 231127113@fzu.edu.cn). Yanxiang Zhu is with VeriMake Innovation Laboratory, Nanjing Renmian Integrated Circuit Company Ltd., Nanjing 210088, China (e-mail: zhuyanxiang@verimake.com). Han Bao is with the department of Electronics and Informatics (ETRO), Vrije Universiteit Brussel, Brussels 1050, Belgium (email: han.bao@vub.be)

FP number with $E = 0$ has no implicit 1 in the significand, so zero and subnormal values can be represented. In addition, exponent $E=2^{e-1}-1$ is reserved for the representation of $\pm\infty$ and $Nans$.

In FP8 quantization, the data range $[2^{-2^{e-1}}, 2^{2^{e-1}}]$ is uniformly divided into m -bit intervals corresponding to contiguous powers of two segment [10]. The quantization step size QS_i for each segment depends on both the exponent of the input $\text{Round}(\log_2 |X_i|)$ and the mantissa bit width m , as formulated below,

$$QS_i = 2^{\text{Round}(\log_2 |X_i|) - m}. \quad (3)$$

Each input element X_i is initially divided by a global scaling factor S . For the S selection, we computed the global dynamic range of activations across layers and determined a unified S using normalization or percentile-based calibration,

$$\text{Clip}\left(\frac{X_i}{S}\right) = \begin{cases} \frac{X_i}{S} & \text{if } \frac{X_i}{S} \in [FP8_{min}, FP8_{max}], \\ FP8_{max} & \text{if } \frac{X_i}{S} > FP8_{max}, \\ FP8_{min} & \text{if } \frac{X_i}{S} < FP8_{min}. \end{cases} \quad (4)$$

The processed value is mapped to a uniform quantization grid across consecutive powers of two. The complete quantization step for each element is summarized in follow,

$$X_i^{(q)} = QS_i \cdot \text{Round}\left(\frac{\text{clip}\left(\frac{X_i}{S}\right)}{QS_i}\right), \quad (5)$$

where $X_i^{(q)}$ is the quantized value.

The global representation of GCNs is given in equation Eq. (6)

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}), \quad (6)$$

where $H^{(l)}$ is the input feature of layer l , $\tilde{A} = A + I_N$ with A the adjacency matrix and I_N the identity matrix. $\tilde{D}$ is the diagonal node degree matrix and $W^{(l)}$ is the trainable layer matrix. The Eq. (3) to Eq. (5) will be implemented for $\tilde{D}$, $\tilde{A}$, $H^{(l)}$ and $W^{(l)}$ to obtain $\tilde{D}_{FP8}$, $\tilde{A}_{FP8}$, $H_{FP8}^{(l)}$ and $W_{FP8}^{(l)}$, respectively. Finally, the global representation of quantized GCNs could be obtained as follow,

$$H_{FP8}^{(l+1)} = \sigma(\tilde{D}_{FP8}^{-1/2} \tilde{A}_{FP8} \tilde{D}_{FP8}^{-1/2} H_{FP8}^{(l)} W_{FP8}^{(l)}). \quad (7)$$

B. Approximate FP8 multiplication

According to the Eq. (1), the FP8 multiplication process of x and y can be represented as:

$$\begin{aligned} \text{Mul}(x, y) &= M_x \cdot 2^{E_x} \times M_y \cdot 2^{E_y} \\ &= (1 + m_x) \cdot 2^{E_x} \times (1 + m_y) \cdot 2^{E_y} \\ &= (1 + m_x + m_y + m_x \cdot m_y) \cdot 2^{E_x + E_y}, \end{aligned} \quad (8)$$

we omit the sign bit as it can be handled through an *XOR* operation. One can note that in Eq. (8), only $m_x \cdot m_y$ involves a multiplication operation for hardware circuit design. The remaining operations can be implemented by using addition or other linear operations such as shift. To alleviate the potential bottleneck caused by mantissa multiplication, Luo et

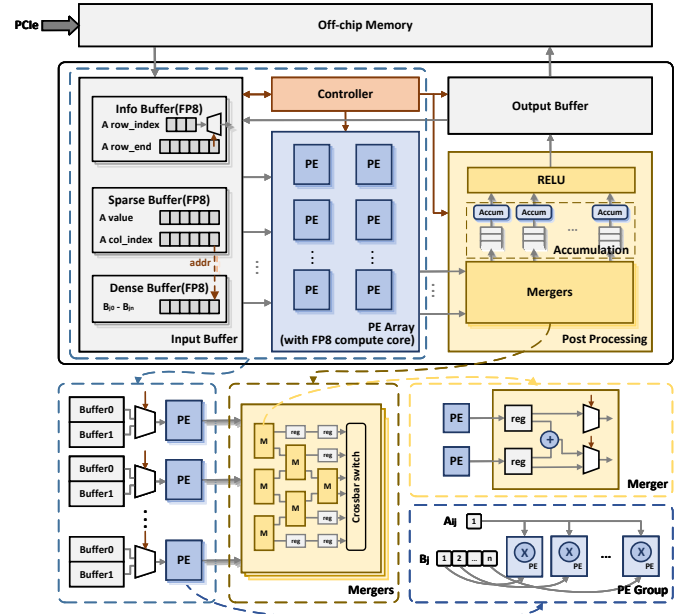


Fig. 1: Overview of the FP8GCN accelerator.

al. [11] propose the *L-Mul* algorithm, which can be designed to approximate the FP8 multiplication process:

$$\begin{aligned} L\text{-Mul}(x, y) &= (1 + m_x + m_y + 2^{-l(m)}) \times 2^{E_x + E_y}, \\ l(m) &= \begin{cases} m & \text{if } m \leq 3, \\ 3 & \text{if } m = 4, \\ 4 & \text{if } m \geq 4, \end{cases} \end{aligned} \quad (9)$$

where m denotes the bit-width of the mantissa. By using this piecewise function approximation, the original multiplication operation can be transformed into shift and addition operations [12].

C. Hardware architecture

1) *Overall*: The architecture of the FP8GCN accelerator is illustrated in Fig. 1. It consists of an input buffer module, a control module, a processing element (PE) array with FP8 approximate multipliers, a post-processing module, and an output buffer module. Prior to execution, feature, weight, and adjacency matrices are preloaded into off-chip memory. Data from off-chip memory is streamed into different types of on-chip input buffers, including info buffer, sparse buffer, and dense buffer. The info buffer stores auxiliary information such as end-of-row flags and row indices associated with the current sparse elements; the sparse buffer stores the non-zero values of the sparse matrix along with their corresponding column indices; and the dense buffer holds partial row values of the dense matrix pointed to by the current column indices. Each PE group is equipped with its own set of the three buffers, and a double-buffering scheme is employed to enable ping-pong buffering, ensuring high pipeline utilization and continuous streaming between on-chip and off-chip memory. The control module generates dataflow control signals based on metadata buffer contents. During execution, the architecture performs Gustavson's algorithm in an element-wise parallel fashion, as illustrated in Fig. 2(a) and (b). Specifically, in each cycle, M

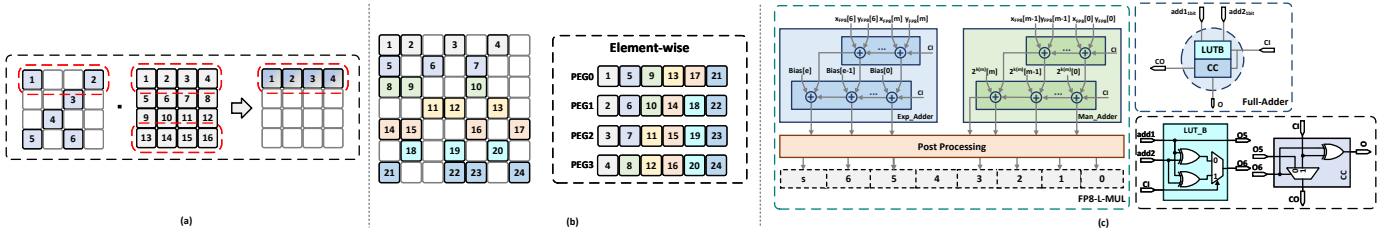


Fig. 2: (a) The matrix multiplication dataflow of Gustavson. (b) Parallelization of element-wise. (c) The fine-grained PE unit design with FP8-based approximate multiplier.

elements of the sparse matrix are fetched in row-major order and distributed to M PE groups. Each PE group performs one multiplication per cycle between a sparse matrix element and the corresponding partial row from the dense matrix. The resulting products are then forwarded to the Mergers for summation and accumulation. The post-processing module comprises three components: Mergers, which combine per-cycle outputs from the PE array; Accumulators, which aggregate results across cycles for the same output row; and a ReLU unit, which applies activation functions to the final outputs. After processing, results are stored in the output buffer. Depending on the control logic, the buffer either writes the data back to off-chip DDR memory or feeds it into the input buffer for further computation.

2) *Unified PE Array*: The core inference procedure of a Graph Convolutional Network (GCN) involves computing the product of three matrices: the adjacency matrix A , the feature matrix X , and the weight matrix W , i.e., performing the operation $A \times X \times W$. This computation can be scheduled in two distinct ways: $(AX)W$ or $A(XW)$. Prior study [13] has shown that choosing the $A(XW)$ execution order significantly reduces the total number of operations—by an average factor of 32 compared to $(AX)W$. Moreover, computing XW first converts the overall GCN inference into a sequence of sparse-dense matrix multiplications. This structure enables the design of a unified Processing Element (PE) architecture. Based on this insight, our accelerator adopts the $A(XW)$ execution strategy and implements it using a uniform PE array. Fig. 2(c) illustrates the PE unit design of the FP8-based approximate multiplier. The multiplier consists of three main components: an exponent adder, a mantissa adder, and a post-processing module. Specifically, the exponent and mantissa adders perform multi-bit addition, corresponding to the respective addition operations in Eq. 9. Both adders share a similar structure, primarily implemented using carry chains (CC) in conjunction with LUT_B to emulate full-adder functionality. The carry chain handles the carry propagation, where the carry-in (CI) signal comes from the previous bit. When operating on the least significant bit, CI is set to 0 for addition and 1 for subtraction. LUT_B is primarily used to implement the functionality of a half-adder. Its output $O6$ corresponds to the sum S in the half-adder. When the least significant bit $CI = 0$, the operation is interpreted as addition: $O6 = add1 \oplus add2$. When $CI = 1$, the operation is interpreted as subtraction, i.e., adding a negative number: $O6 = add1 \oplus (\sim add2)$. The output $O5$ corresponds to the carry-out C in the half-adder. By cascading N full adders, an

N -bit adder can be constructed. The exponent adder consists of an m -bit and an $m+1$ -bit adder, while the mantissa adder consists of an e -bit and an $e+1$ -bit adder. The post-processing module primarily handles sign computation, mantissa carry propagation, and zero multiplication detection.

3) *Mergers and accumulation*: The Merger module consists mainly of adders and selectors, with its operations governed by control signals from the control module. It merges partial products generated within the same cycle from different PE groups corresponding to the same matrix row. Adjacent Merger units are cascaded into a Mergers block, which adopts a multi-stage pipelined structure. This structure synchronously captures M -parallel PE output streams every clock cycle. By applying register retiming, zero-delay data forwarding between pipeline stages is achieved. A crossbar network dynamically aligns the spatial and temporal distribution of partial sums, enabling strictly cycle-synchronized, channelized outputs. Each output channel is connected to a dedicated accumulator unit that performs pipelined aggregation of row-wise partial sums over multiple clock cycles.

III. EXPERIMENTAL RESULTS AND ANALYSIS

A. Experimental setup

1) *Software*: We benchmark the quantization strategy with FP8 and Full precision on four typical datasets (Cora, Citeseer, Pubmed, Yelp) with the software framework PyTorch Geometric (PYG). For a fair comparison, we keep the same network architecture (2 layers), learning rate, training epoch (maximum 200), training strategy (early stop), and optimizer. We adopted this configuration based on findings in prior works [1], [2], which indicate that deeper GCNs often fail to yield accuracy gains and may even degrade performance. We also compare our design with previous 8-bit approximate multipliers by replacing the multiplication units in INT8 and FP8 models and measuring inference accuracy loss.

2) *Hardware*: To evaluate our design, we implement the overall architecture in Verilog HDL and perform synthesis and place-and-route using Vivado 2019.1. The design is deployed on a Kintex-7 K325T FPGA. A total of 16 PE groups operate in parallel, supported by dual-channel DDR (64 bits per channel) to meet bandwidth demands. We adopt a two-layer GCN model with a hidden dimension of 16, and accordingly configure each PE group with 16 multipliers. All experiments are conducted at a clock frequency of 200 MHz. We perform a comparative analysis of existing 8-bit approximate multiplier designs, evaluating their applicability and impact under equivalent conditions. In addition, we use the Intel Xeon 5218 CPU

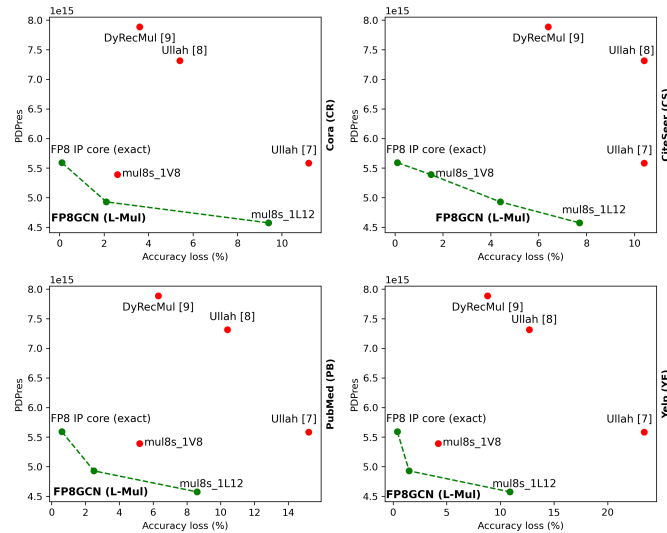


Fig. 3: Pareto frontiers of 8-bit IP cores and approximate multipliers evaluated on different GCN datasets. Accuracy loss is measured with respect to FP32 GCN inference results.

as the baseline for comparison, evaluating against an edge-GPU (NVIDIA Xavier NX), an ASIC design [13], and an edge-FPGA-based accelerator (LW-GCN [2], Kintex-7 K325T FPGA). The execution time of CPU and GPU is obtained from cProfile and NVProf respectively. The power consumption of CPU and GPU is measured by the power meter. The dynamic power of FPGA is obtained from the power reports in Vivado.

B. Experimental Results

We follow the analysis framework of [14], jointly evaluating the trade-off between accuracy loss (measured against FP32 inference) and PPA (PDPres), as shown in Fig. 3. To capture resource-aware cost more comprehensively, we define PDPres as: $PDPres = Power \times CPD \times LUT \times FF \times Carry$. FP8GCN consistently yields the lowest accuracy loss across the most datasets, with an average degradation of only 3%, outperforming all other 8-bit approximate designs. Fig. 3 also demonstrates that mul8s_1L12 delivers the best PPA metrics, but suffers from poor accuracy. FP8GCN, with its low accuracy loss and efficient design, offers the best trade-off and remains Pareto-optimal across all datasets. Furthermore, we conducted comparisons with other platforms, including CPU, GPU, ASIC, and edge-FPGA designs. Parameters of these platforms and the results are listed in Table I. Compared to CPU and edge-GPU, FP8GCN gets speedup up to 44 $\times$ and 30 $\times$, respectively. Compared to ASIC design, leveraging an outer-product-based element-wise computation that minimizes memory traffic, FP8GCN and LW-GCN achieve better performance than GCNAX by avoiding costly write-backs between layers. Although FP8GCN employs fewer PE groups than LW-GCN, its DSP-free implementation enables it to outperform LW-GCN with an average 1.16 $\times$ gain in energy efficiency, making it well-suited for edge deployments.

IV. CONCLUSION

This letter proposes FP8GCN, an edge-FPGA-based GCN inference accelerator that enhances energy efficiency by lever-

TABLE I: Performance Comparison With Other Designs. The Xeon 5218 CPU is Used as the Baseline for Comparison.

	Edge GPU	GCNAX [13]	LW-GCN [2]	FP8GCN
Processor	Xavier NX	ASIC 40nm	K325T	K325T
Frequency	1.1 GHz	1.0 GHz	200 MHz	200 MHz
Resource Utilization				
LUT	-	-	195333 (73%)	130860 (49%)
FF	-	-	94369 (23%)	110248 (27%)
BRAM	-	-	292 (66%)	373.5 (84%)
DSP	-	-	512 (61%)	0
Dynamic Power (W)				
	14.98	4.48	8.16	5.44
Latency [Speedup relative to the CPU baseline]				
CR	1.87ms [1.01 $\times$]	0.2ms [9 $\times$]	41.2 μ s [46 $\times$]	61 μ s [31 $\times$]
CS	1.88ms [2 $\times$]	0.25ms [16 $\times$]	65.2 μ s [60 $\times$]	88.1 μ s [44 $\times$]
PB	2.01ms [6 $\times$]	1.8ms [7 $\times$]	571 μ s [22 $\times$]	639 μ s [20 $\times$]
YE	194.9ms [5 $\times$]	N/A	N/A	72.1ms [12 $\times$]
Energy Efficiency (graph/kJ) [Improvement ratio relative to the CPU baseline]				
CR	3.57E4 [8 $\times$]	1.12E6 [265 $\times$]	2.98E6 [704 $\times$]	3.01E6 [712 $\times$]
CS	3.55E4 [2 $\times$]	8.93E5 [433 $\times$]	1.88E6 [913 $\times$]	2.09E6 [1010 $\times$]
PB	3.32E4 [5 $\times$]	1.24E5 [194 $\times$]	2.14E5 [334 $\times$]	2.88E5 [450 $\times$]
YE	3.43E2 [3 $\times$]	N/A	N/A	2.55E3 [28 $\times$]

aging FP8 quantization and a custom-designed FP8 approximate multiplier. This approach is particularly well-suited for edge and embedded deployment scenarios. Experimental results show that the approximate FP8 GCN model incurs an average accuracy loss of only 3%, outperforming previous 8-bit approximation methods. Compared to CPU and edge GPU platforms, FP8GCN achieves average speedups of 26.8 $\times$ and 14.5 $\times$, respectively. In terms of energy efficiency, FP8GCN delivers up to 1010 $\times$, 84.4 $\times$, and 1.33 $\times$ improvements over CPU, edge GPU, and edge FPGA-based accelerators, respectively. In future work, we plan to explore DSP-compatible hybrid designs and integrate them into a broader range of accelerators.

REFERENCES

- [1] R. Chen *et al.*, “Graph-OPU: A Highly Integrated FPGA-Based Overlay Processor for Graph Neural Networks,” in *IEEE 33rd FPL*. IEEE, 2023, pp. 228–234.
- [2] Z. Tao *et al.*, “LW-GCN: A lightweight FPGA-based graph convolutional network accelerator,” *ACM TRETS*, vol. 16, no. 1, pp. 1–19, 2022.
- [3] B. Zhang *et al.*, “BoostGCN: A Framework for Optimizing GCN Inference on FPGA,” in *IEEE 29th FCCM*. IEEE, 2021, pp. 29–39.
- [4] R. Chen *et al.*, “Ate-gcn: An fpga-based graph convolutional network accelerator with asymmetrical ternary quantization,” in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*. IEEE, 2025, pp. 1–6.
- [5] K. P. Yalamarthi *et al.*, “Low-complexity distributed-arithmetic-based pipelined architecture for an lstm network,” *IEEE TVLSI*, vol. 28, no. 2, pp. 329–338, 2019.
- [6] M. A. Alhartomi *et al.*, “Low-area and low-power vlsi architectures for long short-term memory networks,” *IEEE Jour. Emer. Select. Top. Circu. Syst.*, vol. 13, no. 4, pp. 1000–1014, 2023.
- [7] S. Ullah *et al.*, “Area-Optimized Accurate and Approximate Softcore Signed Multiplier Architectures,” *IEEE TC*, vol. 70, no. 3, pp. 384–392, 2020.
- [8] —, “High-Performance Accurate and Approximate Multipliers for FPGA-Based Hardware Accelerators,” *IEEE TCAD*, vol. 41, no. 2, pp. 211–224, 2021.
- [9] S. Vakili *et al.*, “DyRecMul: Fast and Low-Cost Approximate Multiplier for FPGAs using Dynamic Reconfiguration,” *ACM TRETS*, 2024.
- [10] A. Kuzmin *et al.*, “FP8 Quantization: The Power of the Exponent,” in *NeurIPS*, 2022, pp. 14 651–14 662.
- [11] H. Luo *et al.*, “Addition is all you need for energy-efficient language models,” *arXiv preprint arXiv:2410.00907*, 2024.
- [12] R. Chen *et al.*, “FPGA-Based Approximate Multiplier for FP8,” in *IEEE 33rd FCCM*. IEEE, 2025, pp. 227–235.
- [13] J. Li *et al.*, “Gcnax: A flexible and energy-efficient accelerator for graph convolutional neural networks,” in *IEEE 27th HPCA*. IEEE, 2021, pp. 775–788.
- [14] S. S. Sahoo *et al.*, “Axocs: Scaling fpga-based approximate operators using configuration supersampling,” *IEEE TCAS-I*, vol. 71, no. 6, pp. 2646–2659, 2024.